



CSE 323: Operating System Design

xv6 Operating System Feature Addition

Feature 1: System Call Tracer (Aorpita Rahman Aorpi - 2311676043)

Feature 2: Live CPU Usage Monitor (Anika Mehreen - 2312531042)

GitHub Link

https://github.com/anikaMehreen/CSE323_Project

Submitted By:

Aorpita Rahman Aorpi
Anika Mehreen

2311676043
2312531042

Submitted To:

Farhan Tanvir Khan (FTH)

Section: 04

Abstract

This report is about a project that adds two new features in xv6. The first tool called strace that watches any running program and prints out every time it asks the OS for help. For example, every time it reads a file. This helps to see exactly what a program is doing behind the scenes. The second tool called top, shows a live table of every process running on the system such as what state it is in, how much CPU time it has used, how much memory it occupies and which process created it. It refreshes every second like a dashboard. Everything was tested inside QEMU and worked correctly.

1. Introduction

An operating system (OS) is the software that manages a computer's hardware and lets programs run. xv6 is a tiny OS built by MIT professors for teaching. It works just like a real OS but is small enough that students can read every line of code and understand it. This project extends xv6 by adding two tools that are common in real operating systems like Linux but were missing from xv6.

This project adds two features to xv6:

- **Feature 1- System Call Tracer:** A tool called strace. It watches any program and prints every system call it makes, basically every time the program asks the OS to do something. For example: pid 4: syscall read -> 16
- **Feature 2- Live CPU Usage Monitor:** A tool called top. It shows a live, refreshing table of all running processes such as their process id (PID), parent process id (PPID), memory (bytes), state, CPU ticks and name. Similar to the top command on Linux.

The goals of this project were:

- Add a trace() system call so any process can log which system calls it makes
- Build the strace command so users can trace any program from the terminal
- Add a getprocs() system call to collect information about all running processes
- Build the top command to display a live process table
- Build a makestates program to show all various process states at once

2. Literature Review

2.1 The xv6 Operating System

xv6 was created by Russ Cox, Frans Kaashoek, and Robert Morris at MIT (Cox et al., 2023). It is based on Unix which is one of the first operating systems from the 1970s. xv6 runs on the RISC-

V processor and has only about 6,000 lines of code, making it perfect for learning. It handles all the basics: running processes, managing memory, and reading/writing files.

2.2 What is a System Call?

A system call is how a user program asks the OS for help. Programs cannot touch hardware directly rather they must request it through the OS. Examples include `read()` to read a file, `write()` to print text, and `fork()` to create a child process. xv6 handles system calls through a central dispatch table where each call has a unique number (Cox et al., 2023).

2.3 The Linux strace Tool

Linux has a tool called `strace` that shows every system call a program makes (man7.org, 2023). Developers use it daily to debug programs and understand how they work. Our Feature 1 does the same thing inside xv6 using a simpler method, a bitmask stored inside the process record instead of Linux's more complex `ptrace()` mechanism.

2.4 Process Monitoring and the top Command

Linux has a command called `top` that shows all running processes in a live display (Kerrisk, 2010). It reads data from a special part of Linux called `/proc`. Since xv6 has no such system, Feature 2 adds a dedicated system call `getprocs()` to collect the same kind of information and send it safely to the user.

2.5 Process States

Every process is always in exactly one state. Tanenbaum and Bos (2015) describe this as a core concept of process management.

2.6 Bitmasks

A bitmask is an integer where each bit acts as a yes/no switch. Feature 1 uses a bitmask to choose which system calls to trace. For example, bit 5 = system call number 5 = `read()`. Setting bit 5 means 'trace read calls'. The number 32 equals bit 5 turned on ($2 \text{ to the power of } 5 = 32$). Bitmasks are used here because checking them takes just one CPU instruction which is very fast.

3. Methodology

3.1 Feature 1: System Call Tracer

How it works:

- When a user runs `strace 32 ls`, here is exactly what happens:
- `strace` reads the mask (32) and calls the `trace()` syscall, which saves 32 in the process's record
- `strace` then calls `exec()` to replace itself with `ls` — but the trace mask stays saved

Now `ls` is running. Every time `ls` makes a syscall, the kernel checks: is that syscall's bit set in the mask?

- If yes, the kernel prints: `pid 4: syscall read -> 16`
- When `ls` forks children, they also inherit the mask, so they get traced too

The key change in the kernel: logging

Inside `kernel/syscall.c`, after every syscall finishes, one check was added:

```
if(p->tracemask & (1 << num))  
    printf("pid %d: syscall %s -> %ld\n", p->pid, syscall_names[num], p->trapframe->a0);
```

This runs after the syscall has already returned, so it shows the real result. The check is one bitwise AND — extremely fast, no performance impact.

The key change for fork — inheritance

Inside `kernel/proc.c`, one line was added so children inherit tracing:

```
np->tracemask = p->tracemask;
```

The `strace` command

The user program `strace.c` does three things:

- Reads the mask number from the command line
- Calls `trace()` to set it
- Calls `exec()` to hand control to the target program — tracing is already active

3.2 Feature 2: Live CPU Usage Monitor

When a user runs `top`, here is exactly what happens every second:

- `top` calls `getprocs()` this crosses into the kernel
- The kernel loops through all 64 process slots in the process table
- For each slot it copies the safe data (pid, name, state, ticks, memory) into user memory
- `top` prints a table row for each active process
- `top` waits 100 ticks then repeats forever

Counting CPU time

To measure how much CPU each process uses, a counter called `cputicks` was added to every process's record. Every time the scheduler gives a process a CPU turn, this counter goes up by 1. This happens right before the actual switch:

```
p->cputicks++;           // count this turn
swtch(&c->context, &p->context); // now give the CPU to the process
```

The safe data structure

The kernel's full process record (`struct proc`) contains sensitive internal data that user programs should never see. So a simpler, safe version was created:

```
struct procinfo {
    int  pid;    // process ID
    char name[16]; // program name
    int  state;  // sleeping, running, etc.
    uint64 cputicks; // CPU turns received
    int  ppid;   // parent's PID
    uint64 sz;   // memory size in bytes
};
```

Sending data to user space

The kernel cannot just hand a pointer to user memory rather they are separate spaces. `copyout()` is the safe bridge that copies data across. `sys_getprocs()` uses it for every process slot it fills.

The makestates demo program

“makestates” creates one process in every possible state so `top` can display them all.

4. Results and Findings

4.1 Feature 1: strace Worked Correctly

Test run:

```
$ strace 32 ls
```

Mask 32 = bit 5 = SYS_read. Every read() call that ls made was printed. The output showed one line per file in the filesystem, with return value 16 (bytes read), and a final 0 at end of directory:

```
pid 4: syscall read -> 16
.      1 1 1024
pid 4: syscall read -> 16
README  2 2 2425
pid 4: syscall read -> 16
top      2 26 35272
pid 4: syscall read -> 0
```

Fork inheritance was also tested by running strace 32 sh and typing ls inside. The child shell also produced trace output, confirming the mask is passed to child processes correctly.

4.2 Feature 2: top Showed All CPU Usage Monitor Status

After running “makestates”, the top output looked like this:

PID	PPID	SZ	STATE	TICKS	NAME
1	0	16384	sleeping	26	init
2	1	20480	sleeping	16	sh
5	4	16384	zombie	2	makestates
4	1	16384	runnable	24	makestates
6	4	16384	runnable	14	makestates
7	4	16384	runnable	17	makestates
8	4	16384	running	13	makestates
9	4	16384	running	13	makestates
10	4	16384	sleeping	17	makestates
11	2	16384	running	6	top

5. Conclusion and Recommendations

5.1 Conclusion

Both features were successfully built, tested, and merged. The project started with a version of xv6 that had no way to observe what programs were doing or what processes were running. It now has two working tools that answer both of those questions.

Feature 1 makes the invisible visible by showing every conversation between a program and the OS in real time. Feature 2 gives a live dashboard of everything happening on the system at any moment.

5.2 Recommendations

A few ideas for how this project could be improved in the future:

Show syscall arguments in strace: Right now only the return value is shown. Showing the input too like the filename being read would make debugging much more useful.

Sort processes in top: Let the user sort by CPU ticks or memory so the most active process appears first.

Reference:

- Cox, R., Kaashoek, M. F., & Morris, R. (2025). xv6: A simple, Unix-like teaching operating system. MIT PDOS. <https://pdos.csail.mit.edu/6.828/2025/xv6/book-riscv-rev5.pdf>
- MIT PDOS. (2023). xv6-riscv source code. GitHub. <https://github.com/mit-pdos/xv6-riscv>
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2021). Operating system concepts (10th ed.). John Wiley & Sons.
- <https://github.com/remzi-arpacidusseau/ostep-projects>.
- <https://pages.cs.wisc.edu/~remzi/OSTEP/lab-projects-xv6.pdf>
- Kerrisk, M. (2010). The Linux programming interface. No Starch Press.
- Man7.org. (2023). strace(1) Linux manual page. <https://man7.org/linux/man-pages/man1/strace.1.html>
- Patterson, D., & Waterman, A. (2019). The RISC-V reader: An open architecture atlas. Strawberry Canyon LLC.
- Tanenbaum, A. S., & Bos, H. (2015). Modern operating systems (4th ed.). Pearson.